Deliverable 3: Final Features, Documentation, and Comparison Report

Course: Cross-Language Application Development

Project: Expense Tracker Application

Languages Used: Python and C++

Student Name:

Date:

Introduction

This project aimed at creating an Expense Tracker application in two programming languages Python and C++. The goal was to illustrate the same application under various different program environments with the same core functionality. Application lets users create, manage, search and delete expenses and total the amount of money spent. The development of the application in two languages gave hands on experience in software development, problem solving and learning the pros and cons of both the languages.

Application Completion

The project was successfully finished for both Python and C++. The two implementations will have the same basic functionality that the assignment requires. They can record new expenses with the date, amount, category and expense description. The applications also enable users to view all their saved expenses, search through their expenses with keywords, delete selected expenses and see the total value of all of their expenses recorded.

The Tkinter library is used in the implementation to create a graphical user interface (GUI). This version has buttons, text boxes, drop-down menus, and a table showing all expenses for the user to interact with the application. The expenses are recorded in a JSON file ensuring data is available even if the app is shut down.

The C++ implementation has been created as a console application in Visual Studio Community. It offers the same functionality but in a menu based interface. The user enters into the application by choosing from the menus and typing in the necessary information directly into the console. The expenses are stored in a text file with the resulting data being preserved between runs of the program.

Comparison Between Python and C++

Both the applications do the same work, but the implementation of the two programs is different from Python and C++. The syntax of Python was easy to learn and learn to program with, and it had a lot of standard libraries available. Using the Tkinter library, it was fairly simple to create a pleasing GUI without the need for extra programs. In addition, Python also makes it easy to handle files and process JSON, which means there's less code to use and fewer lines of code in the project. The whole implementation in Python was less bulky and simpler to debug.

C++ needed to be programmed in a more detailed way in contrast. Careful management of variables, data structures and memory usage are required. The handling of files would involve more coding, and developing a GUI interface would need more frameworks like Qt or Windows Forms alternatives, which are more difficult to setup. Therefore, it was decided to write the C++ version as a console application to still be able to perform all necessary tasks.

C++ has an edge in one aspect: performance. C++ is a compiled language and it processes faster and uses less memory than Python due to its execution as a machine code language. But it's not noticeable for an application this size and Python is simpler to develop. Python's automatic garbage collection helps minimise the risk of memory-related programming mistakes, while C++ offers developers more control over memory management. This control is

to enable highly optimized programs, but it also makes programs more complex and the likelihood of bugs being introduced is higher if memory is not managed properly.

Challenges Encountered

There were a few hurdles in the way of creating this project. The first problem was to set up the C++ development environment. The correct compiler and Visual Studio setup were needed and needed to be configured prior to a successful compile of the application.

The other challenge was to introduce persistent storage. There are two applications: one using JSON files and the other using C++ with manual reading and writing to text files. Extensive testing was carried out to ensure that the expenses records were being saved and loaded properly. It has taken extra work to develop the graphical interface too. The Python GUI is created using Tkinter library and incorporated with buttons, input fields and a table to display expenses records. Input validation has been added to make sure the user has entered the date in the proper format (YYYY-MM-DD) and positive numbers when entering the amount of the expense. There was also a search feature added, so they would be able to search expenses by category or description.

Testing and Results

Various expense records were used to test both implementations. All major functions were found to be working properly by testing. Users could make an addition to their expenses, view the records they've saved, search by keyword, view running totals of expenses, and remove selected records without impacting any other data. Invalid dates and incorrect numerical values were not accepted as input. Testing also showed that expense data was still available even when the applications were restarted by using persistent storage files.

GitHub Repository

The complete source code, documentation for the project, README and supporting reports have been added to the project GitHub repository.

GitHub Repository:

https://github.com/nadhikari48742/ExpenseTracker

Conclusion

This project illustrated the successful implementation of the same application in two different programming languages and with the same functionality. Because of its ease of use and the availability of numerous libraries, it was decided to use Python to build a user-friendly graphical application.Python was chosen for the ease of use and the availability of numerous libraries to build a user-friendly graphical application. C++ needed more programming time, but offered more control of the system resources and better execution speed. Making both versions enhanced the understanding of the concepts of programming, software design, handling of files, debugging and implementation methods specific to the programming language. Overall, the project was considered as a success due to the ability to meet all the requirements of the assignment and valuable experience in cross language software development.

Screenshots (Insert Before Submission)

Figure 1: Python Expense Tracker GUI

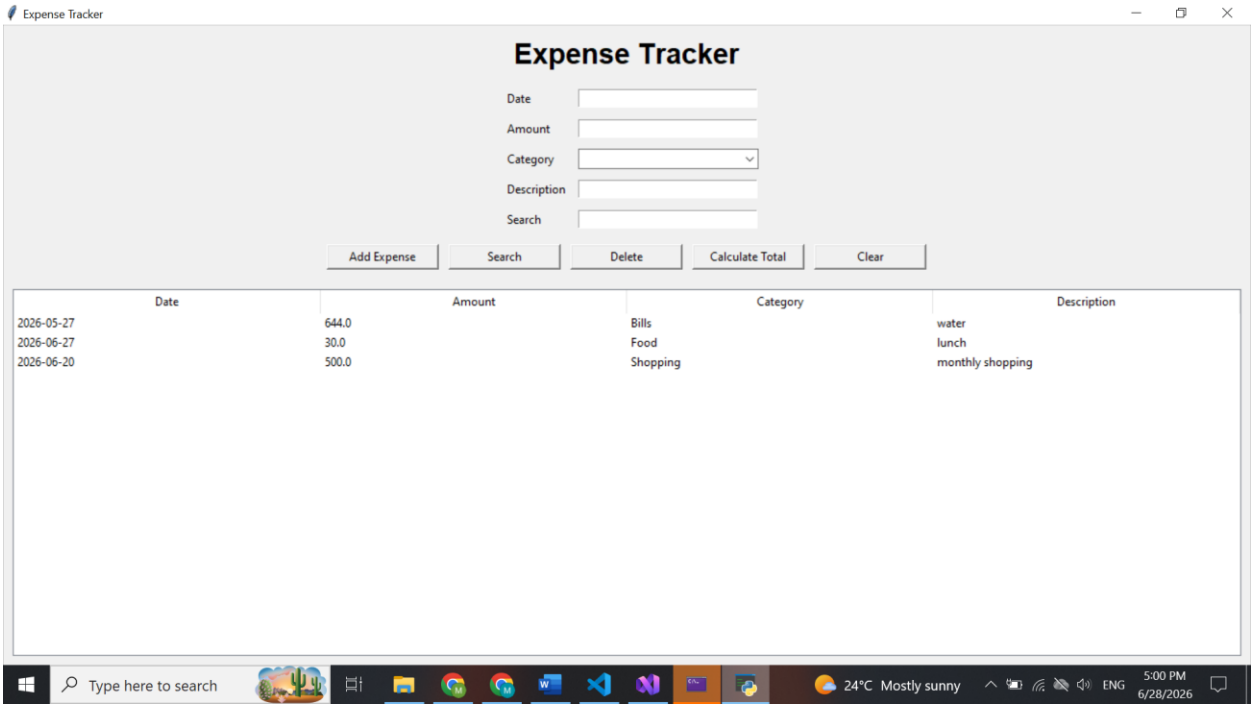


Figure 2: C++ Expense Tracker Console Application

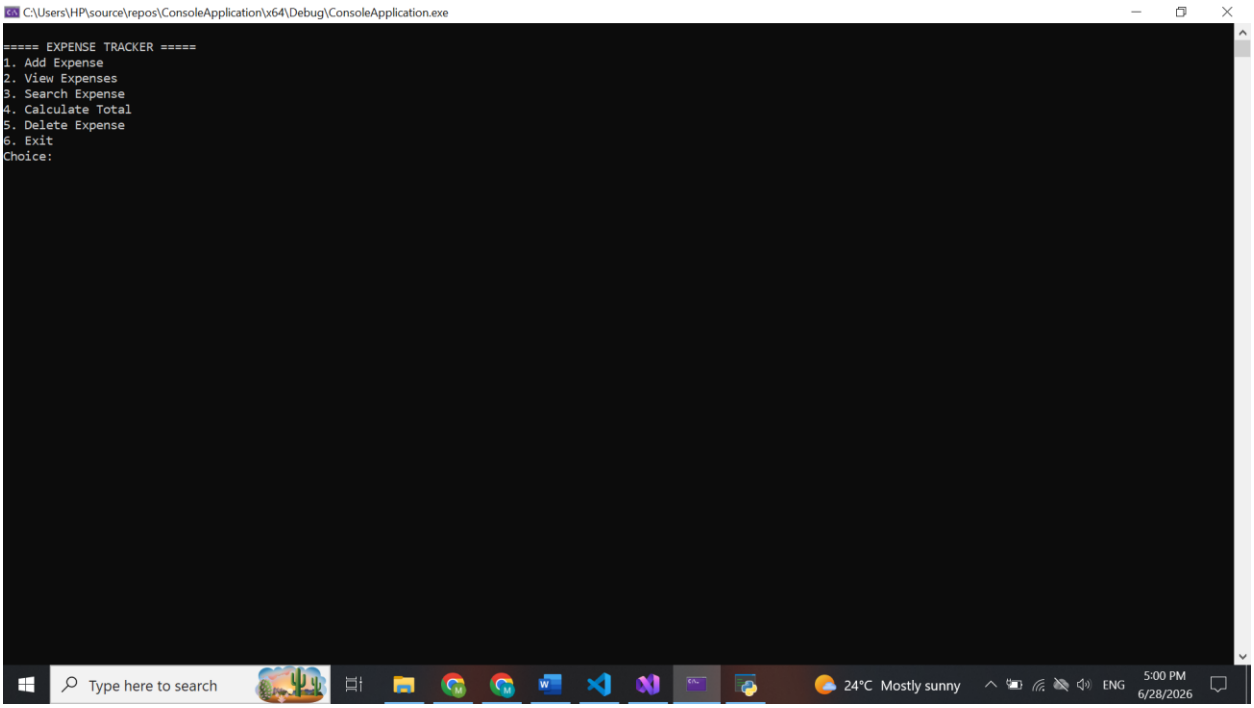
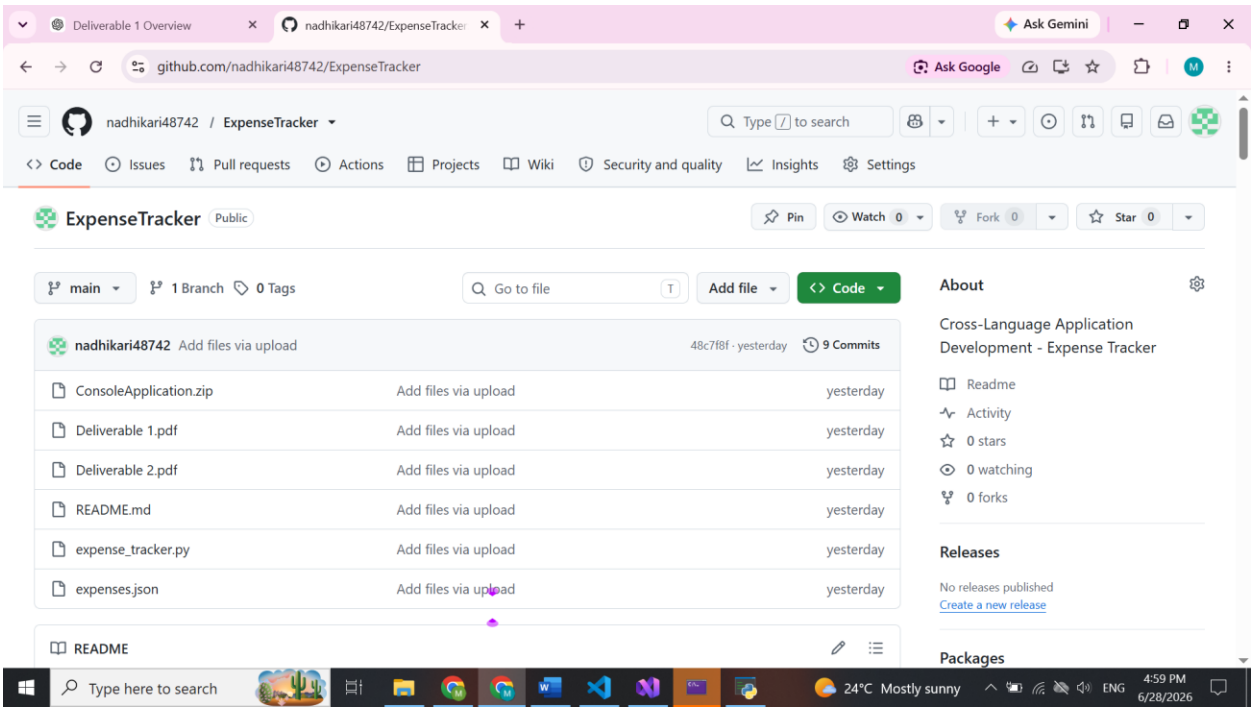


Figure 3: GitHub Repository


Deliverable 1 Overview
nadhikari48742/ExpenseTracker
Ask Gemini
github.com/nadhikari48742/ExpenseTracker
Ask Google
nadhikari48742 / ExpenseTracker
Type / to search
Code
Issues
Pull requests
Actions
Projects
Wiki
Security and quality
Insights
Settings
ExpenseTracker
Public
Pin
Watch 0
Fork 0
Star 0
main
1 Branch
0 Tags
Go to file
Add file
Code
nadhikari48742 Add files via upload
48c7f8f · yesterday
9 Commits
ConsoleApplication.zip Add files via upload yesterday
Deliverable 1.pdf Add files via upload yesterday
Deliverable 2.pdf Add files via upload yesterday
README.md Add files via upload yesterday
expense_tracker.py Add files via upload yesterday
expenses.json Add files via upload yesterday
README
About
Cross-Language Application Development - Expense Tracker
Readme
Activity
0 stars
0 watching
0 forks
Releases
No releases published
Create a new release
Packages
Type here to search
24°C Mostly sunny
ENG
4:59 PM
6/28/2026